

CS336 Spring 2025 Assignment 2: Systems Writeup

Contents

1 Problem: benchmarking_script	2
2 Problem: nsys_profile	2
3 Problem: mixed_precision_accumulation	4
4 Problem: benchmarking_mixed_precision	5
5 Problem: memory_profiling	5
6 Problem: pytorch_attention	6
7 Problem: torch_compile	8
8 Problem: flash_forward	9
9 Problem: flash_backward	9
10 Problem: flash_benchmarking	9
11 Problem: distributed_communication_single_node	9
12 Problem: naive_ddp	9
13 Problem: naive_ddp_benchmarking	9
14 Problem: minimal_ddp_flat_benchmarking	9
15 Problem: ddp_overlap_individual_parameters	9
16 Problem: ddp_overlap_individual_parameters_benchmarking	9
17 Problem: ddp_overlap_bucketed	9
18 Problem: ddp_bucketed_benchmarking	10
19 Problem: communication_accounting	10
20 Problem: optimizer_state_sharding	10

1 Problem: benchmarking_script

(a) Deliverable

Implemented in `cs336_systems/benchmark.py`. The script initializes the model from CLI hyperparameters, creates random input data, runs warmup steps, then measures forward-only or forward+backward timing with `timeit.default_timer()` and `torch.cuda.synchronize()`.

(b) Forward/backward timings (5 warmup, 10 measured steps)

Model	Fwd Mean (ms)	Fwd Std (ms)	Bwd Mean (ms)	Bwd Std (ms)	Step Mean (ms)	Step Std (ms)
small	105.303	38.286	62.392	30.468	170.464	38.242
medium	184.545	38.394	85.389	23.722	288.648	25.349
large	339.047	2.485	156.582	1.572	499.472	3.688
xl	733.624	42.155	313.152	1.035	1099.266	5.916
2.7b	838.264	9.618	457.806	1.187	1307.808	24.517

Table 1: Part (b) benchmark results on CUDA (A100 40GB).

Forward and backward time increase with model size. Variability is generally low for larger models in this run (especially backward), while small/medium show higher standard deviation.

(c) Effect of fewer/no warmup steps

Model	Warmup 0		Warmup 1		Warmup 2		Warmup 5	
	Mean	Std	Mean	Std	Mean	Std	Mean	Std
small	225.844	138.148	169.072	36.043	176.863	30.199	170.464	38.242
medium	344.245	181.775	294.368	34.751	295.297	25.260	288.648	25.349
large	596.205	239.325	570.078	35.475	518.251	32.955	499.472	3.688
xl	1186.364	289.365	1077.503	31.389	1092.626	12.791	1099.266	5.916
2.7b	1402.911	225.944	1321.353	34.838	1319.389	37.589	1307.808	24.517

Table 2: Part (c): step total timing (ms), 10 measured steps, varying warmup count.

Without warmup ($w=0$), both average step time and variance are much worse across sizes. Using 1–2 warmup steps substantially improves stability, but can still differ from the 5-step baseline because one-time startup effects (kernel autotuning/selection, allocator behavior, cache setup, lazy initialization) may not be fully amortized yet.

2 Problem: nsys_profile

Profiling Coverage

We profiled with `nsys` at context lengths {128, 256, 512, 1024} for each model size {small, medium, large, xl, 2.7b}, using one profiled step after warmup. As allowed by the handout, several larger

configurations ran out of memory and are noted below.

Model	128	256	512	1024
Forward-only				
small	yes	yes	yes	yes
medium	yes	yes	yes	yes
large	yes	yes	yes	OOM
xl	yes	yes	yes	OOM
2.7b	yes	yes	OOM	OOM
Train step (fwd+bwd+AdamW)				
small	yes	yes	yes	yes
medium	yes	yes	yes	yes
large	yes	yes	yes	OOM
xl	OOM	OOM	OOM	OOM
2.7b	OOM	OOM	OOM	OOM

Table 3: Nsight coverage; OOM indicates CUDA out-of-memory during warmup/profiled step.

(a) Forward-pass time and comparison to Python timers

Model (ctx=128)	Nsight forward (ms)	Part 1.1.3 forward mean (ms)	Delta (ms)
small	201.262	105.303	+95.959
medium	406.419	184.545	+221.874
large	781.562	339.047	+442.515
xl	1298.431	733.624	+564.807
2.7b	1209.294	838.264	+371.030

Table 4: Nsight forward-pass NVTX timing vs. Part (b) Python benchmark timings.

The Nsight forward-pass times are substantially higher than the Python benchmark values in this run. This mismatch is expected because Nsight instrumentation introduces overhead and here we profiled single iterations rather than averaging 10 timed steps like in Part (b).

(b) Dominant CUDA kernel and invocation count

In the forward pass, the highest cumulative-GPU-time kernel is generally a GEMM kernel (for example, `ampere_sgemm_128x64_tn` for small/medium/large at context length 128). For small@128 forward, this top kernel was invoked 25 times in one forward pass. The dominant kernel is not always the same once backward is included (e.g., small@128 train-step is dominated by a different GEMM variant), so the answer is configuration-dependent.

Here, a kernel name like “ampere sgemm 128x64 tn” denotes an Ampere-optimized single-precision matrix-multiplication kernel. The 128x64 part indicates tile shape, and tn indicates transpose/non-transpose layout choices for GEMM operands.

(c) Non-matmul kernels with non-trivial runtime

Beyond GEMMs, non-trivial forward runtime comes from elementwise and reduction kernels, including direct copy/masked fill/indexing and reduction-style kernels used by normalization and softmax-related computation. These appear consistently in the top non-GEMM entries across model sizes.

(d) Training-step vs inference kernel mix

On configurations where both modes fit in memory, matrix-multiplication kernels account for about 80.0% of CUDA kernel time in forward-only runs versus about 71.4% in full training steps. So compared to inference, training shifts more time into non-matmul work (backward elementwise/reduction kernels and optimizer multi-tensor update kernels).

(e) Softmax vs matmul within self-attention

Using an additional attention-annotated forward profile (medium@128), the summed attention softmax runtime was 48.61 ms, while attention matrix multiplications were 30.31 ms (QK) + 21.94 ms (PV) = 52.25 ms. This runtime gap is much smaller than the FLOP gap (matmuls are far higher FLOPs), consistent with softmax and related steps being more memory-bound and less compute-efficient than GEMMs.

3 Problem: mixed_precision_accumulation

(a) Accumulation experiments

We evaluated the four accumulation snippets from the prompt and obtained:

Case	Final value of <code>s</code>
<code>float32</code> accumulator + <code>float32</code> addend	10.0001335
<code>float16</code> accumulator + <code>float16</code> addend	9.9531250
<code>float32</code> accumulator + <code>float16</code> addend	10.0021362
<code>float32</code> accumulator + <code>cast(float16 addend→float32)</code>	10.0021362

Table 5: Observed outputs from the four mixed-precision accumulation snippets.

(b) Why this happens

Accumulating directly in FP16 is much less accurate because FP16 has far fewer mantissa bits, so repeated additions incur larger rounding error. Using an FP32 accumulator significantly improves stability, but if each addend is first quantized to FP16 then that quantization error is still present, which is why cases (3) and (4) match each other and remain less accurate than pure FP32 accumulation.

4 Problem: benchmarking mixed_precision

(a) Dtypes under FP16 autocast

Assuming model parameters are initialized in FP32 and training is done on CUDA with FP16 autocast (for example, `torch.autocast(...)` configured with `dtype=torch.float16`):

- Model parameters inside autocast: FP32 (autocast does not permanently cast parameters).
- Output of `ToyModel.fc1`: typically FP16.
- Output of `ToyModel.ln`: typically FP32 (layer norm is on autocast’s FP32-safe path).
- Model logits (`ToyModel.fc2` output): typically FP16.
- Loss: typically FP32 for common PyTorch loss ops under autocast.
- Gradients: FP32 (matching FP32 parameter storage dtype).

(b) Why layer norm is treated differently

Layer norm computes per-feature statistics (mean/variance) and involves reductions plus subtract/square/rsqrt operations that are numerically sensitive to precision. FP16 can lose significant accuracy (and may overflow/underflow more easily), so running layer norm in FP32 is safer. BF16 has a wider exponent range than FP16 (so overflow is less problematic), but it still has limited mantissa precision, so FP32 treatment of normalization remains helpful for stability.

(c) FP32 vs BF16 mixed-precision timings

Model	Fwd FP32 (ms)	Fwd BF16 MP (ms)	Bwd FP32 (ms)	Bwd BF16 MP (ms)	Step FP32 (ms)	Step BF16 MP (ms)
small	87.552	110.179	71.047	59.873	158.618	170.065
medium	217.868	223.842	75.005	96.604	292.888	320.462
large	384.381	363.331	163.561	134.228	547.968	497.575
xl	781.901	514.076	323.489	180.433	1105.413	694.529
2.7b	841.362	352.867	456.769	129.098	1298.162	481.981

Table 6: Forward/backward/step means with and without BF16 autocast mixed precision (warmup=5, measured=10, ctx=128).

Mixed precision showed little or negative benefit for the smallest models in this run, but produced clear speedups for larger models. Step-time speedup grew with scale (about $1.10\times$ at large, $1.59\times$ at xl, and $2.69\times$ at 2.7b), consistent with larger workloads benefiting more from Tensor Core throughput and amortizing autocast overhead.

5 Problem: memory_profiling

(a) Memory-profiler instrumentation and timelines

I added memory-profiler support to the `nsys_profile_workload.py` script using PyTorch CUDA memory history recording and snapshot export, plus peak-memory JSON export. Snapshots were generated under `results/memory/` for successful runs, including separate files for mixed precision on and off at context length 128.

From the successful forward profiles, the timeline is consistent with a large rise during the forward region and then a plateau near peak reserved memory after kernel completion. For full train-step runs (forward+backward+optimizer), all 2.7B configurations on this 40GB GPU ran out of memory during optimizer-state allocation, so a complete train-step timeline could not be produced in this environment.

(b) Peak memory by context length

Context length	Forward peak allocated (GiB, FP32)	Full train-step peak (GiB, FP32)
128	17.940	OOM
256	24.176	OOM
512	OOM	OOM

Table 7: Peak memory from profiler JSON outputs for the 2.7B model.

(c) Mixed-precision peak memory

For BF16 autocast mixed precision, forward-pass peak allocated memory was 22.321 GiB (ctx=128), 26.410 GiB (ctx=256), and 36.724 GiB (ctx=512). Full train-step runs with BF16 autocast still OOM at all tested context lengths (128/256/512). In this setup, mixed precision did not significantly reduce memory footprint for the 2.7B model and did not make full train steps fit on a 40GB GPU.

(d) Residual-stream activation size

For one residual-stream activation tensor in FP32, size is

$$\begin{aligned}
 &= \text{batch} \times \text{context} \times d_{\text{model}} \times 4 \text{ bytes} \\
 &= 4 \times L \times 2560 \times 4.
 \end{aligned}$$

So this is $40960L$ bytes, i.e. 5 MB at $L=128$, 10 MB at $L=256$, and 20 MB at $L=512$ (using $\text{MB} = 1024^2$ bytes).

(e) Largest allocations at reduced detail

Inspecting the saved snapshot data for forward runs shows largest active allocations in large chunks (e.g., up to ~ 100 MB blocks in the allocator snapshot, with frequent 40 MB blocks). The available stack traces in this environment resolve mainly to the CUDA caching allocator internals, so the coarse interpretation is that these largest blocks correspond to persistent model/activation storage managed by the allocator during large forward GEMM-heavy regions.

6 Problem: pytorch_attention

(a) Forward/backward benchmark grid

I benchmarked the baseline PyTorch attention implementation with batch size fixed to 8, no head dimension, $d_{\text{model}} \in \{16, 32, 64, 128\}$, and sequence length $L \in \{256, 1024, 4096, 8192, 16384\}$. For

each configuration, I ran 10 warmup iterations, then timed 100 synchronized forward passes and 100 synchronized backward passes (the backward timer starts right before `backward()` and excludes setup forward compute). I also recorded allocated CUDA memory right after forward and before backward on the first measured iteration.

d_{model}	L	Fwd mean (ms)	Bwd mean (ms)	Memory before bwd (GiB)	Status	Note
16	256	0.291	1.118	0.021	ok	–
16	1024	0.820	1.686	0.082	ok	–
16	4096	6.708	17.722	1.032	ok	–
16	8192	25.385	65.899	4.048	ok	–
16	16384	–	–	–	OOM	CUDA OOM
32	256	0.239	1.148	0.022	ok	–
32	1024	0.588	1.564	0.086	ok	–
32	4096	7.042	18.060	1.047	ok	–
32	8192	26.656	67.335	4.079	ok	–
32	16384	–	–	–	OOM	CUDA OOM
64	256	0.241	1.203	0.024	ok	–
64	1024	0.623	1.632	0.094	ok	–
64	4096	7.591	18.695	1.079	ok	–
64	8192	28.789	69.722	4.141	ok	–
64	16384	–	–	–	OOM	CUDA OOM
128	256	0.243	1.124	0.028	ok	–
128	1024	0.681	1.664	0.110	ok	–
128	4096	8.596	19.871	1.141	ok	–
128	8192	32.905	74.290	4.267	ok	–
128	16384	–	–	–	OOM	CUDA OOM

Table 8: PyTorch attention benchmark grid (A100 40GB, FP32).

The first OOM appears at $L = 16384$ for every tested d_{model} , which indicates memory is dominated by the L^2 attention-score/probability tensors rather than the $L \cdot d_{model}$ projections.

Memory accounting and scaling

Using the smallest failing setting ($B=8$, $L=16384$, $d_{model}=16$, FP32), one $B \times L \times L$ tensor requires:

$$8 \times 16384 \times 16384 \times 4 = 8,589,934,592 \text{ bytes} \approx 8 \text{ GiB}.$$

Naive attention materializes large $L \times L$ intermediates (scores/probabilities), and backward needs saved activations plus additional gradient/intermediate buffers, so memory pressure rises quickly. Since these dominant tensors scale as $O(BL^2)$, saved-for-backward memory grows quadratically with sequence length: doubling L increases this memory by about $4\times$. This matches the measured trend from about 1.0–1.1 GiB at $L=4096$ to about 4.0–4.3 GiB at $L=8192$.

To eliminate this L^2 memory cost, avoid materializing the full attention matrix. Use tiled attention plus recomputation checkpointing so backward does not keep full score/probability matrices in memory.

7 Problem: torch_compile

(a) Compiled vs uncompiled PyTorch attention

I extended the attention benchmark script to optionally run `torch.compile` on the attention function (`--compile-attention`). Settings match Problem `pytorch_attention`: batch size 8, no head dimension, $d_{model} \in \{16, 32, 64, 128\}$, $L \in \{256, 1024, 4096, 8192, 16384\}$, 10 warmup steps, 100 measured steps, synchronized timing.

d_{model}	L	Fwd uncompiled (ms)	Fwd compiled (ms)	Bwd uncompiled (ms)	Bwd compiled (ms)	Status
16	256	0.291	0.185	1.118	0.524	ok
16	1024	0.820	0.365	1.686	1.073	ok
16	4096	6.708	3.659	17.722	10.638	ok
16	8192	25.385	14.570	65.899	38.358	ok
16	16384	—	—	—	—	OOM (both)
32	256	0.239	0.202	1.148	0.723	ok
32	1024	0.588	0.497	1.564	1.213	ok
32	4096	7.042	4.735	18.060	11.350	ok
32	8192	26.656	17.349	67.335	40.527	ok
32	16384	—	—	—	—	OOM (both)
64	256	0.241	0.200	1.203	0.702	ok
64	1024	0.623	0.484	1.632	1.110	ok
64	4096	7.591	5.317	18.695	11.957	ok
64	8192	28.789	19.508	69.722	42.997	ok
64	16384	—	—	—	—	OOM (both)
128	256	0.243	0.213	1.124	0.644	ok
128	1024	0.681	0.529	1.664	1.177	ok
128	4096	8.596	6.311	19.871	13.146	ok
128	8192	32.905	23.594	74.290	47.812	ok
128	16384	—	—	—	—	OOM (both)

Table 9: Compiled vs uncompiled attention timings (A100 40GB, FP32).

Compiled attention was consistently faster on all non-OOM configurations. Forward speedups were about $1.14\times$ to $2.25\times$, and backward speedups were about $1.29\times$ to $2.13\times$. The OOM boundary did not change: both versions failed at $L = 16384$ for all tested d_{model} .

(b) Compiled vs uncompiled end-to-end Transformer

I added `--compile-model` and `--include-optimizer-step` to `cs336.systems/benchmark.py`, then benchmarked at context length 128, batch size 4, warmup 5, measured 10, comparing vanilla vs compiled model for forward-only and full train step (forward+backward+AdamW step).

Model	Fwd uncompiled (ms)	Fwd compiled (ms)	Train-step uncompiled (ms)	Train-step compiled (ms)	Status
small	169.652	11.464	167.638	42.937	ok
medium	360.032	35.021	341.366	128.684	ok
large	679.762	72.261	625.880	284.403	ok
x1	1207.474	147.182	OOM	573.523	uncompiled train OOM
2.7b	1101.367	231.768	OOM	OOM	train OOM (both)

Table 10: End-to-end Transformer benchmark: vanilla vs `torch.compile` (A100 40GB, FP32).

In this environment, compiling the full model produced large forward-pass improvements (about $4.75\times$ to $14.8\times$ across sizes). For full train steps including optimizer updates, compiled runs were also faster where both fit ($\sim 2.2\times$ to $3.9\times$). An additional observation is that compiled x1 train-

step fit while uncompiled x1 OOMed at optimizer step initialization; both compiled and uncompiled 2.7b train-step OOMed.

8 Problem: flash_forward

TODO: Add implementation notes and requested deliverables.

9 Problem: flash_backward

TODO: Add backward implementation results and validation.

10 Problem: flash_benchmarking

TODO: Add FlashAttention benchmarking tables and analysis.

11 Problem: distributed_communication_single_node

TODO: Add distributed communication measurements and response.

12 Problem: naive_ddp

TODO: Add naive DDP implementation deliverable details.

13 Problem: naive_ddp_benchmarking

TODO: Add naive DDP benchmarking results and discussion.

14 Problem: minimal_ddp_flat_benchmarking

TODO: Add flat-buffer DDP benchmarking results.

15 Problem: ddp_overlap_individual_parameters

TODO: Add implementation and correctness evidence.

16 Problem: ddp_overlap_individual_parameters_benchmarking

TODO: Add overlap benchmarking results and conclusions.

17 Problem: ddp_overlap_bucketed

TODO: Add bucketed overlap implementation details.

18 Problem: ddp_bucketed_benchmarking

TODO: Add bucket-size benchmarking results and analysis.

19 Problem: communication_accounting

TODO: Add communication-volume accounting calculations.

20 Problem: optimizer_state_sharding

TODO: Add implementation details and validation.

21 Problem: optimizer_state_sharding_accounting

TODO: Add memory/speed accounting and ZeRO comparison.